

Inspections and Static Analysis

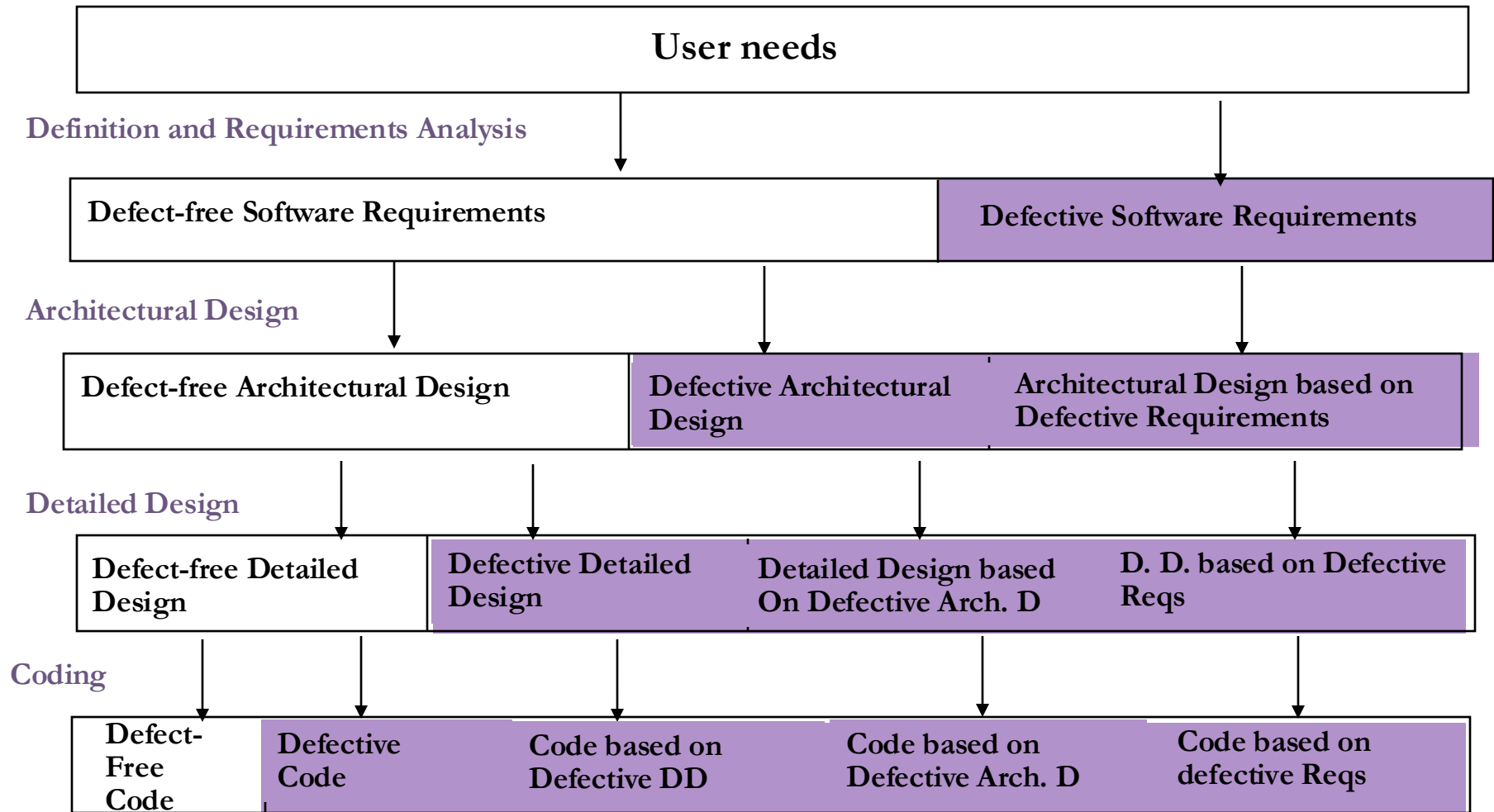
DASS , Spring 2025
IIIT Hyderabad

Motivation

- ▶ Detecting errors late in the development cycle is expensive
 - ▶ E.g. A requirements defect that is found only at testing costs almost 100 times more to fix than if it had been found and fixed at requirements itself
 - ▶ Need to rework not only the requirements doc, but all other deliverables produced from it: change the design, change the code, rerun tests!
 - ▶ The earlier in the lifecycle we find problems, the cheaper they are to fix



Error Introduction and Propagation

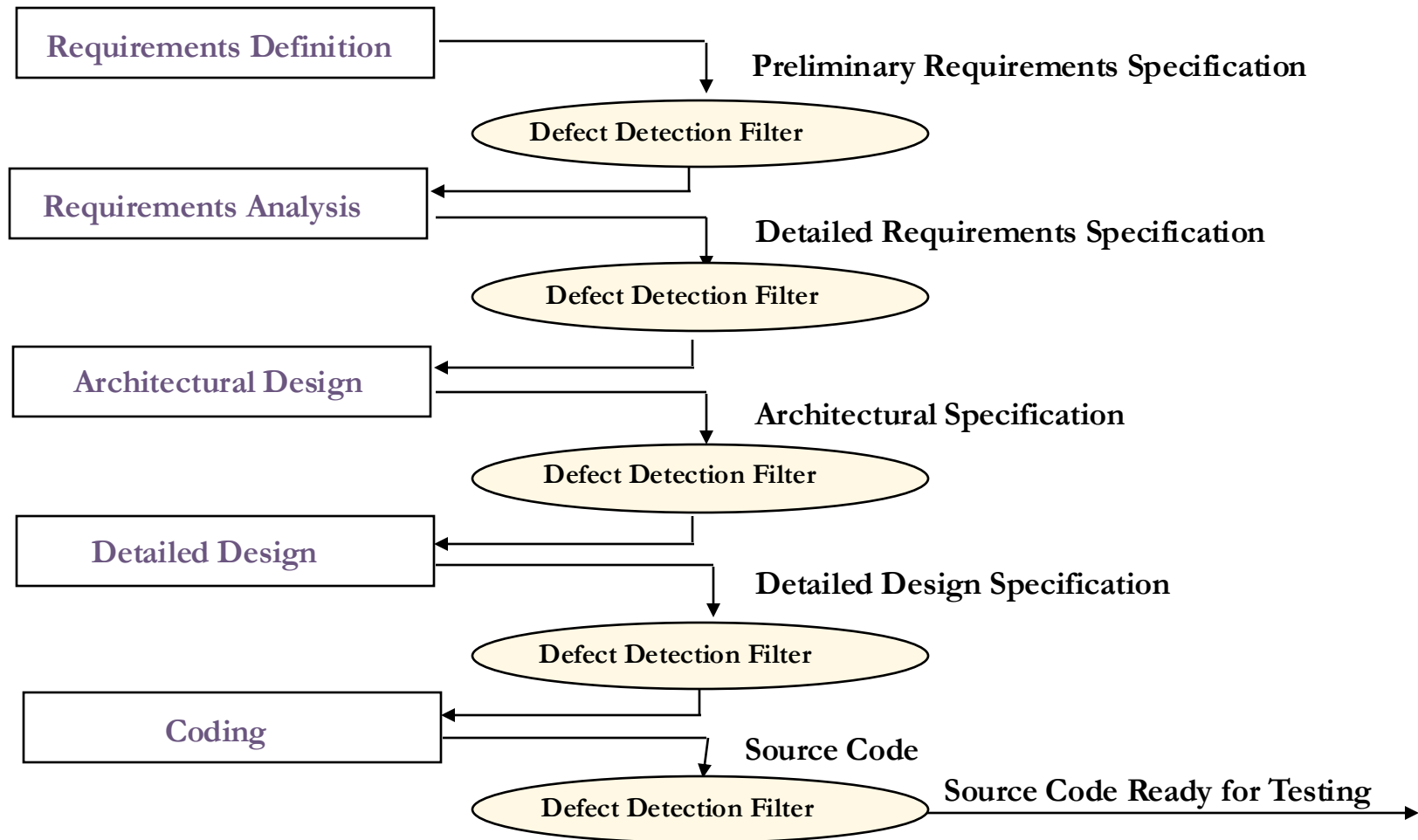


Motivation (2)

- ▶ Multiple stages of defect removal
 - ▶ If we inspect each deliverable (requirements, design, code), and then do multiple stages of testing (unit tests, integration tests, system tests), then we get many chances to find defects
- ▶ Like filtering multiple times: the result is much cleaner!



Multiple Stages of Defect Removal



Inspections & Static Code Analysis

Inspections

- Inspections are formal or informal evaluations of code, aimed at finding defects and improving code quality through collective review.

Static Code Analysis

- Static code analysis refers to the automated analysis of source code without executing it, focusing on detecting potential errors, code smells, and security vulnerabilities

Both practices enhance code reliability, facilitate adherence to coding standards, and reduce development costs by catching issues early

Inspections often rely on human judgment, enabling team collaboration and knowledge sharing, while **static code analysis** provides objective metrics and identifies issues that might be overlooked.



Current processes used for inspections

- ▶ **Code Reviews:**
 - ▶ Peer review process where developers examine each other's code.
 - ▶ Uses tools like GitHub, Bitbucket.
- ▶ **Pair Programming:** Two developers collaborate on a single piece of code, allowing real-time feedback and inspection.
- ▶ **Formal Inspections:** A structured evaluation involving multiple stakeholders, including developers, QA, and project managers.
- ▶ **Checklists:** Utilizing predefined lists of potential issues or best practices to guide reviews (e.g., consistency, readability).
- ▶ **Tools for Static Analysis:** Utilize various tools that automate code inspections
 - ▶ SonarQube
 - ▶ Checkstyle
 - ▶ PMD
 - ▶ FindBugs/SpotBugs



Static Code Analysis - Examples

```
public void process (String input) {  
    if (input.length() > 0) {  
        System.out.println (input);  
    }  
}
```

Possible null pointer exception



Static Code Analysis - Examples

```
public double calculateArea (double radius) {  
    return Math.PI * radius * radius;  
}
```

Use a constant instead of a Magic number



Static Code Analysis - Examples

```
import java.util.List; public class ItemProcessor {  
  
    public void processItems(List<Item> items) {  
  
        for (int i = 0; i < items.size(); i++) {  
            Item item = items.get(i);  
            System.out.println(item.getName());  
        }  
    }  
}
```

Method uses index-based access, which is less efficient compared to using a for-each loop



Static Code Analysis - Examples

```
public void calculateSum (int a, int b) {  
    System.out.println (a + b);  
}
```

```
public void calculateTotal (int x, int y) {  
    System.out.println (x + y);  
}
```

Duplicate logic can be refactored



Static Code Analysis - Examples

```
public List<User> getUsers (String userType) {  
    String query = "SELECT * FROM users WHERE type = '" + userType + "'";  
}
```

Vulnerability Identification. - SQL injection



Static Code Analysis - Examples

```
import java.io.FileReader;

public class FileManager {
    public void readFromFile (String filename) {
        FileReader fr = new FileReader(filename);
    }
}
```

Not closing the FileReader properly can lead to resource leak

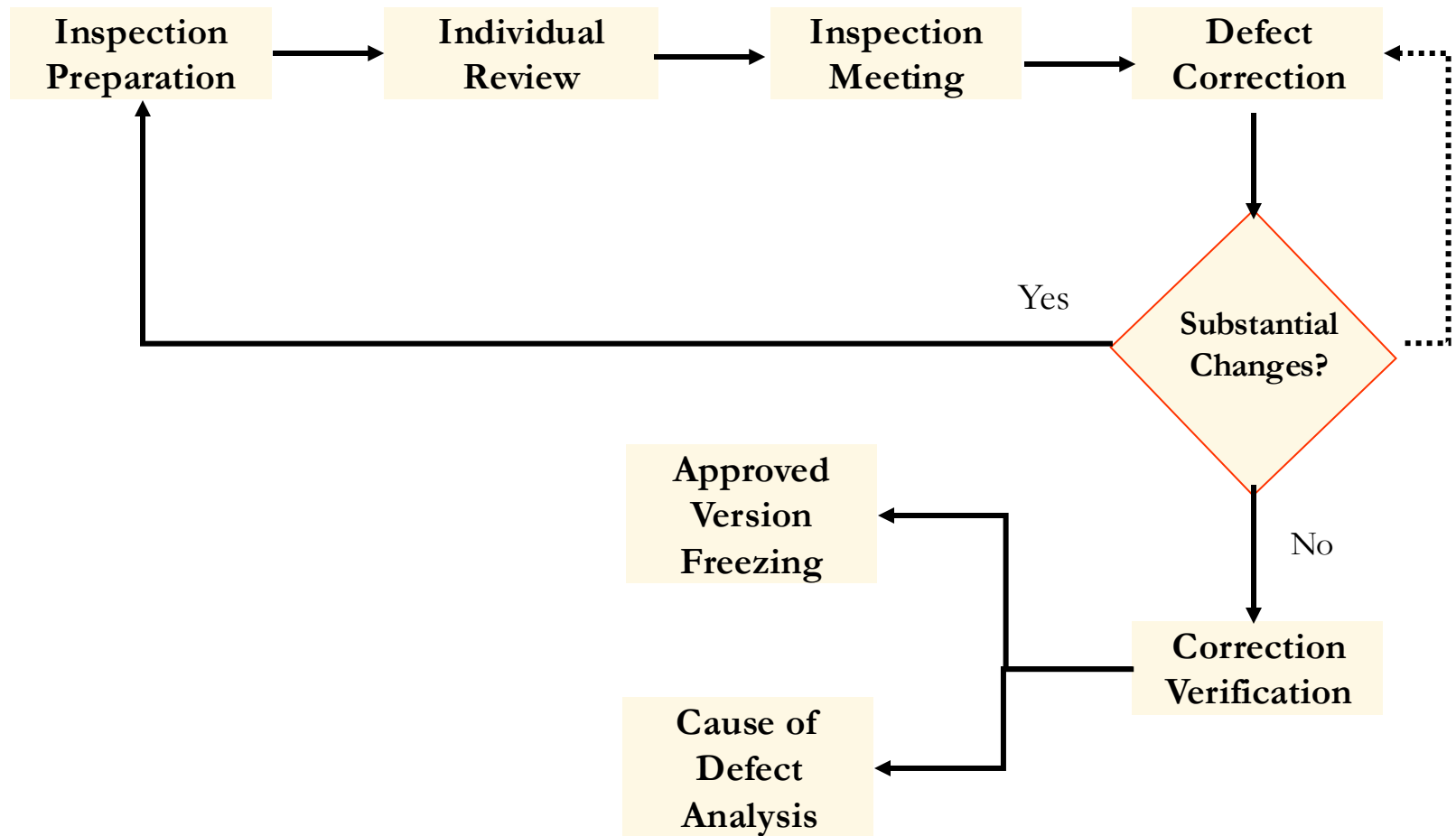


Inspections & its effectiveness

- A study showed that, as a rule of thumb, each defect identified during inspection saves around 9 hours of time downstream.
- AT&T found inspections led to 14% increase in productivity and tenfold increase in quality.
- HP found 80% of the errors detected during inspections were unlikely to be caught by testing.
- HP, Shell Research, Bell Northern, and AT&T all found inspections 20 to 30 times more efficient at testing in detecting errors.
- IBM found inspections gave a 23% increase in productivity and a 38% reduction in bugs detected after unit test.



Inspection Phases



Formal inspection process

- ▶ Defined roles: Author, Reader, Moderator, Scribe, Inspectors
- ▶ **Author** - distributes the artifact that he/she has coded
- ▶ **Reader** interprets the code for the inspectors
 - ▶ If reader is different from author, reduces possibility of author propagating their own misunderstandings
- ▶ **Inspectors** prepare comments before meeting, provide their inputs and contribute to discussions during meeting
- ▶ **Moderator** keeps the discussions on track, also responsible for checking later that the problems found have been fixed
- ▶ **Scribe** ensures that problems found get recorded



Code Inspection Report

Team / Project	<Team name / Project name in Black.>
Component	<Identification of inspected component in Black.>
Date	<Date in black (e.g., Jan 21, 2008) in Black.>
Author	<Name of the component's primary author in Black.>
Moderator	<Name of inspection moderator in Black.>
Reader	<Name of inspection reader in Black.>
Scribe	<Name of inspection scribe in Black.>
Other Inspectors	<Names of other inspectors in Black.>

Defect No.	Line #	Severity H,M,L	Description
1			
2			
3			
4			
5			
6			
7			
8			
9			
10			
11			
12			
13			
14			
15			
16			
17			
18			
19			
20			
21			
22			
23			
24			
25			
26			
27			
28			
29			
30			

```

1  /*
2  * PrintSpooler - Models a print spooler using a StringBuffer. Allows the user to get
3  *               an instance of the spooler and add strings or characters. The
4  *               contents of the spooler can then be sent to the system output
5  *               device.
6  *               Note that PrintSpooler is a Singleton - only a single instance
7  *               of it can exist at anytime. The method getPrintSpooler insures
8  *               this by only performing one instance creation and then simply
9  *               returning the same object reference to subsequent requests.
10 */
11 public class PrintSpooler {
12
13     private static PrintSpooler spooler = new PrintSpooler();    // the one and only
14                                                                    // instance of PrintSpooler
15     public StringBuffer spoolBuffer = null;                      // model using a StringBuffer
16
17     public PrintSpooler(){                                         // constructor for object creation
18         };
19
20     public PrintSpooler getPrintSpooler(){                        // only allow one-time creation of
21         PrintSpooler mySpooler = spooler;                        // PrintSpooler, else return referece,
22         if ( spooler == null)
23             mySpooler = new PrintSpooler();
24         return mySpooler;
25     }
26
27     public void addString( String s){                             // add a String to the spooler
28         spoolBuffer.append( s );
29     }
30
31     public void addChar( char theChar, int length){              // add a character, allow repeating
32         for ( int i=1; i<length ; i++)                          // of same character
33             spoolBuffer.append( theChar);
34     }
35
36     public void print(){                                           // send to system.out device, add header info
37         spoolBuffer.insert(0, "PrintSpooler - Version 1.3 Header ");
38         System.out.print(spoolBuffer);
39     }
40
41     public void clear(){                                           // clear the spooler
42         spooler = new PrintSpooler();
43     }
44 }

```

```

1  /*
2  * PrintSpooler - Models a print spooler using a StringBuffer. Allows the user to get
3  *               an instance of the spooler and add strings or characters. The
4  *               contents of the spooler can then be sent to the system output
5  *               device.
6  *               Note that PrintSpooler is a Singleton - only a single instance
7  *               of it can exist at anytime. The method getPrintSpooler insures
8  *               this by only performing one instance creation and then simply
9  *               returning the same object reference to subsequent requests.
10 */
11 public class PrintSpooler {
12
13     private static PrintSpooler spooler = new PrintSpooler(); null;    // the one and only
14                                         // instance of PrintSpooler
15     public private StringBuffer spoolBuffer = null;                    // model using a StringBuffer
16
17     public private PrintSpooler(){                                     // constructor for object crea
18         spoolBuffer = new StringBuffer();
19     };
20
21     public static PrintSpooler getPrintSpooler(){                      // only allow one-time creation of
22         PrintSpooler mySpooler = spooler;                            // PrintSpooler, else return referece,
23         if ( spooler == null)
24             mySpooler spooler = new PrintSpooler();
25         return mySpooler spooler;
26     }
27
28     public void addString( String s){                                     // add a String to the spooler
29         spoolBuffer.append( s );
30     }
31
32     public void addChar( char theChar, int length){                     // add a character, allow repeating
33         for ( int i=1; i<length i<=length; i++)                       // of same character
34             spoolBuffer.append( theChar);
35     }
36
37     public void print(){                                                 // send to system.out device, add header info
38         spoolBuffer.insert(0, "PrintSpooler - Version 1.3 Header"); // make a constant
39         System.out.print(spoolBuffer);
40     }
41
42     public void clear(){                                                 // clear the spooler
43         spooler = new PrintSpooler();
44         spoolBuffer = new StringBuffer();
45     }

```

Other benefits of Inspections

- ▶ Team members get familiar with the code
 - ▶ Backup if someone is unavailable
- ▶ More uniform design and coding practices across team
- ▶ Knowledge sharing
- ▶ Shared understanding & improved communication
 - ▶ Identify miscommunication and misperceptions
- ▶ More perspective on how everything comes together

